AIQA — Truthfulness Cleanup, Step 1: Application Source Connection

Scope: Replace the Test Case Generator's fake, simulated "Connected" status (Gap 1 of the Phase 1–6 Final Consolidated Audit) with a genuine, bounded, SSRF-guarded backend connectivity check. No other gap (Step 2–5, RBAC Testing UI) was touched.

Date: 2026-08-26 **Branch:** `claude/run-this-9eeb2e`

---

1. What was fake before

`ApplicationSourceSection.tsx`'s `handleConnect` did client-side URL-syntax validation only, waited a fixed `setTimeout(600ms)`, then unconditionally reported:

- `status: 'CONNECTED'`
- `httpStatus: 200` (hardcoded, literal)
- `sslValid: true` (hardcoded, literal, shown even for `http://` URLs)
- `pingMs: Math.random() * 150 + 20` (fabricated)

for **any** syntactically valid URL — including hosts that don't exist, hosts that refuse connections, and internal/metadata addresses. No network request was ever made. This was flagged **CRITICAL** in the Phase 1–6 Final Consolidated Audit (`AIQA_PHASE_1_TO_6_FINAL_CONSOLIDATED_AUDIT.md`, Gap 1).

2. What is real now

Backend: `services/orchestrator/src/app_source.rs` *(new)*

- `POST /api/v1/projects/:project_id/verify-application-source` (Operator role, project-scoped) -> `verify(url)`.
- `verify()`: format/scheme check -> `executor::validate_target_url` (the **same, unmodified** SSRF guard used by API Testing and Performance Testing — RFC1918/link-local/documentation/multicast/unspecified ranges always blocked; loopback allowed **only** on port 8080, the orchestrator's own real port) -> `attempt_real_request()`.
- `attempt_real_request()`: a real `reqwest` GET with `redirect::Policy::none()` (prevents redirect-based SSRF bypass), a bounded 10s timeout, response body never read. Classifies the real outcome via `executor::classify_reqwest_error` (widened from `fn` to `pub(crate) fn` to allow reuse — no behavior change to that function).
- Five real, distinct outcomes, each truthfully labeled: `CONNECTED` (any real HTTP response, 2xx–5xx alike — reachability, not application success), `UNREACHABLE` (real DNS/connect/TLS/timeout failure), `REJECTED` (blocked by the SSRF policy, generic message — no internal detail leaked), `INVALID_URL` (bad format/scheme), `ERROR` (the verification request itself failed).
- No SSL/certificate-validity claim is made. `reqwest`'s API exposes no certificate detail, and a broken cert fails the request outright (classified `TLS_FAILED` -> `UNREACHABLE`) — so the old `sslValid: true` claim is gone, not replaced with a different fabrication.
- Real, measured `latency_ms` via `std::time::Instant`, only ever the actual elapsed wall-clock time of the real request attempt.
- Audited: every call writes a real `audit()` row (`action: "app_source.verify"`).

Frontend: `forentend/src/components/test-generator/ApplicationSourceSection.tsx` *(rewritten)*

- `handleConnect` is now `async`, calls `aiqaApi.appSource.verify(projectId, { url })`, and renders exactly what the backend returns.
- Seven genuinely distinct UI states: `IDLE` / `CHECKING` / `CONNECTED` / `UNREACHABLE` / `REJECTED` / `INVALID_URL` / `ERROR`.
- `CHECKING` is tied to the real in-flight promise, not a fixed delay.
- The "Connected Readout" shows only backend-returned `httpStatus` / `latencyMs` — nothing fabricated, nothing shown if the backend didn't return it.
- A backend/network failure while calling the endpoint itself renders `ERROR`, never a silent fallback to `CONNECTED`.
- No project selected -> `Connect` is disabled with a truthful reason shown in the UI; no request is sent.
- `Disconnect` resets to the real `IDLE` state (not a fabricated one).

3. Files changed

```
forentend/src/components/pages/TestCaseGeneratorPage.tsx          |   9 +-
forentend/src/components/test-generator/ApplicationSourceSection.tsx | 137 ++++++++++++++++-----
forentend/src/services/aiqaApi.ts                                 |  15 +++
forentend/src/types.ts                                            |  30 +++--
forentend/src/types/contract.ts                                   |  27 ++++
services/orchestrator/src/api.rs                                  |  37 ++++++
services/orchestrator/src/api_tests.rs                            | 144 ++++++++++++++++++++---
services/orchestrator/src/engine.rs                               |  53 +++++---
services/orchestrator/src/executor.rs                             |   2 +-
services/orchestrator/src/lib.rs                                  |   1 +
```

New files: `services/orchestrator/src/app_source.rs`, `forentend/src/components/test-generator/ApplicationSourceSection.test.tsx`.

`engine.rs`'s change is **not** part of the Application Source feature itself — see §5, it is a necessary regression fix discovered while verifying this work, disclosed in full below rather than folded silently into the feature diff.

4. Test evidence

Backend — `cargo test --lib`, run 4 consecutive times from a clean environment (no manually-started server, `netstat` confirmed port 8080 free before each run)

Run	Result
1	`314 passed; 0 failed; 2 ignored`
2	`314 passed; 0 failed; 2 ignored`
3	`314 passed; 0 failed; 2 ignored`
4 (post-cleanup, after live verification, port re-confirmed free)	`314 passed; 0 failed; 2 ignored`

314 = the pre-existing 298 (last known-clean baseline, re-confirmed independently in §5) + 16 new (11 `app_source` unit tests + 5 `app_source_verify_*` integration tests). No pre-existing test was deleted, skipped, or weakened.

New integration tests (`api_tests.rs`): `app_source_verify_a_real_reachable_target_reports_connected`, `app_source_verify_a_disallowed_target_reports_rejected_through_the_real_endpoint`, `app_source_verify_a_malformed_url_reports_invalid_url`, `app_source_verify_rbac_viewer_forbidden_operator_allowed`, `app_source_verify_project_isolation_enforced`.

New unit tests (`app_source.rs`, 11): cover format/scheme rejection, SSRF-blocked ranges, a real reachable 2xx target, a real reachable non-2xx target (still `CONNECTED`), a real dead-port target (`UNREACHABLE`), a real timeout target (`UNREACHABLE`), and the safe/sanitized rejection message.

Frontend

- `npx tsc --noEmit`: clean.
- `npx vitest run`: **412 passed / 412** (23 test files), including the new `ApplicationSourceSection.test.tsx` (11/11).
- `npx vite build`: production build succeeds (`✓ built in 20.74s`); the pre-existing >500kB chunk-size warning is unrelated and unchanged.

5. Regressions found during verification, root-caused, and fixed

Running the full backend suite immediately after adding the Step 1 tests surfaced **two** genuine, reproducing failures unrelated to the Application Source feature's own logic. Both are disclosed here in full, per the requirement to report truthfully rather than optimize for a clean-looking diff.

5a. Self-contained test fixture's server died mid-suite (`phase4_api_test_*`, 3 tests, reproduced 2/2)

Symptom: `phase4_api_test_executes_a_real_request_and_passes_a_real_status_assertion`, `phase4_api_test_assertion_mismatch_genuinely_fails_not_a_fake_pass`, and `phase4_api_test_query_parameters_and_headers_genuinely_reach_the_real_request` — the exact three tests the prior "backend test self-containment" fix made reliable — started failing with `CONNECTION_FAILED: error sending request for url (http://127.0.0.1:8080/health)`, reproducing on both of two consecutive full-suite runs.

Root cause: the fixture (`ensure_local_health_target`) used `tokio::spawn` inside a `tokio::sync::OnceCell::get_or_init`, called from whichever `#[tokio::test]` happened to reach it first. Each `#[tokio::test]` function gets its **own**, independent Tokio runtime that is torn down — aborting anything spawned on it — the instant that one test function returns. The fixture's real axum server was therefore only alive for as long as the *first calling test's own* runtime happened to still be executing, which was accidentally "good enough" under the old test mix and its scheduling, and broke the instant the mix changed (this session added 16 new tests, shifting `cargo test`'s scheduling).

Fix (`services/orchestrator/src/api_tests.rs`): the fixture now spawns the real axum server on a **dedicated OS thread** with its own long-lived Tokio runtime that nothing ever tears down, synchronized via a `std::sync::mpsc` channel so the first caller genuinely waits for the real bind (or a real `AddrInUse` fallback) before proceeding — never an arbitrary sleep standing in for a readiness signal. `std::sync::Once` replaces `tokio::sync::OnceCell` since the "start it once" step is itself synchronous.

Verification: 4/4 subsequent full-suite runs clean (see §4).

5b. Approve endpoint could race its own dispatch loop and report a stale/wrong state (`approval_survives_a_process_restart_and_can_still_be_approved`, reproduced 2/2 under full-suite load, 0/8 in isolation)

Symptom: after fixing 5a, the full suite (not this test in isolation — 8/8 clean alone) started failing with `left: "PARTIAL", right: "RUNNING"` — the approve endpoint's response occasionally reported a state other than the `RUNNING` it had just caused.

Root cause (pre-existing, in `engine.rs`, unrelated to Application Source): `approve_and_spawn` called `spawn_run`, which fires a `tokio::spawn`'d background task that sets the run to `Running` **inside** that spawned task — then `approve_and_spawn` returns immediately, without waiting for that transition to actually happen. The HTTP handler then reads the run's state back to build its response. Under enough concurrent load (exactly what 16 additional tests running in parallel supplies), the background task can run far enough ahead — for a plan with a single, zero-delay simulated task, all the way to a terminal state — before the handler reads it back, so the response can show a stale `PENDING_APPROVAL` or an already-progressed state instead of the `RUNNING` the approve call is documented to return.

Fix (`services/orchestrator/src/engine.rs`): extracted the "become Running" transition (in-memory state, durable state, started timestamp, `RunScheduled` event) into a new `mark_run_running()` helper, made idempotent (a no-op if the run is already `Running`). `approve_and_spawn` now calls and **awaits** it synchronously, before handing off to the background dispatch loop — so by the time the HTTP handler reads the run back, `Running` is already guaranteed, both in memory and durably. `spawn_run`'s own internal call to the same helper is now a harmless, idempotent no-op for the approval path, and remains the sole source of the transition — unchanged — for every one of its ~10 other, non-HTTP call sites in `engine_tests.rs`, none of which needed or received any change.

Verification: confirmed absent on the pre-Step-1 baseline commit (4/4 clean runs via `git stash`, 298/298 each time) before being fixed here, then 4/4 clean full-suite runs afterward (see §4) — including the specific full-suite conditions

that reproduced it.

Why these are disclosed rather than silently folded in: neither touches Application Source logic. Both are genuine, pre-existing latent bugs that this session's added test load was the first thing to reliably expose. Leaving them unfixed would have left `cargo test` red for reasons entirely disconnected from this task's actual scope, which the brief's "Backend tests clean" requirement does not permit.

6. Live verification (real backend, real UI, real network)

Real `cargo run` orchestrator on `0.0.0.0:8080`, real `npm run dev` frontend on `localhost:3000`, a scratch admin API key minted directly in the dev sqlite database for this session only and deleted immediately afterward (same technique used in prior verification passes this session). Driven through the actual rendered UI (Test Case Generator -> Application Sources), not just `curl`.

Scenario	Target	UI result	Backend response
LIVE VERIFIED — real reachable target	`http://127.0.0.1:8080/health` (the orchestrator's own real health route — the one documented SSRF-allowed loopback exception)	`Connected` · `http://127.0.0.1:8080/health` · `HTTP 200 • Response time: 1ms`	`{"status":"CONNECTED","reachable":true,"http_status":200,"latency_ms":1}`
LIVE VERIFIED — SSRF-blocked target	`http://169.254.169.254/latest/meta-data/` (cloud metadata range)	`Rejected` · "This target is not allowed by the connection security policy."	`{"status":"REJECTED","reachable":false,"http_status":null,"latency_ms":null}`
LIVE VERIFIED — real unreachable target	`http://1.1.1.1:8081/` (real public IP, closed port)	`Checking` while in flight, then `Unreachable` after a real ~10s timeout	`{"status":"UNREACHABLE","reachable":false,"latency_ms":10001,"message":"The connection attempt timed out."}`
(incidental) loopback on a non-8080 port	`http://127.0.0.1:59999/`	`Rejected`	Confirms the loopback exception is scoped to port 8080 only, as designed — not a defect.
(incidental) RFC 5737 documentation range	`http://192.0.2.1/`	`Rejected`	Confirms `is_documentation()` ranges are correctly blocked — not a defect.

`INVALID_URL` and RBAC-forbidden (`Viewer` role) are **TEST VERIFIED** (automated integration/component tests listed in §4), not re-driven live, since they add no further truthfulness signal beyond what the live-verified network-outcome path already proves.

Cleanup performed: orchestrator process stopped, frontend preview server stopped, scratch API key deleted from the dev database, temporary scratch files removed, port 8080 re-confirmed free before the final backend test confirmation run.

7. Post-fix fake/stub audit

Grepped every new/changed file for `Math.random`, `setTimeout`-as-verification, hardcoded status/SSL literals, `TODO`/`FIXME`/`unimplemented!`/`todo!`. The only two matches are both inside doc comments *describing the removed fake behavior for context* (`ApplicationSourceSection.tsx` line 7, `TestCaseGeneratorPage.tsx` line 146) — no executable fake logic remains. `app_source.rs` has zero matches.

8. What this does not claim

- No SSL/TLS certificate validity is asserted, ever — see §2.
- `CONNECTED` means "a real HTTP response was received," not "the target application is healthy" — a real 404/500 is still `CONNECTED`, truthfully, since reachability and application-level correctness are different questions (proven by a dedicated test).
- This step does not touch Document Upload (Step 2), pre-seeded generated cases (Step 3), Settings disconnection (Step 4), Settings token status (Step 5), or RBAC Testing UI — all explicitly out of scope and untouched.

9. Final Verdict

FIXED.

- No fake success path remains in the Application Source connection feature.
- Every status shown to the user is traceable to a real backend network attempt, through the same SSRF-guarded infrastructure the rest of the system already relies on.
- The two regressions this work's added test load exposed are genuine, pre-existing, unrelated latent bugs (not part of Application Source) that have been root-caused, fixed, and reproducibly verified clean (4/4) — required because the brief's "Backend tests clean" condition does not admit a red suite regardless of cause.
- Backend: 314/314 passing, 4/4 consecutive clean runs. Frontend: 412/412 passing, clean `tsc`, clean production build. Live-verified end-to-end for the connected, SSRF-rejected, and genuinely-unreachable cases.